

Prompting Patterns — Quick Reference

These patterns work whether you're in Plan mode (designing) or Agent mode (building). Use them to get better output from the first prompt.

1. Constraints Up Front

State what you don't want before Cursor picks an approach.

```
Build a data fetching layer for the mCase config browser.  
Use the MCP tools (schema, datalist_by_name) – do NOT make direct HTTP calls to  
mCase.  
Do NOT add a state management library; use React's built-in useState and  
useEffect.
```

Without constraints: Cursor picks the approach. That approach may not be yours.

2. Show Me the Data Shape First

Before writing rendering code, verify the data is right.

```
Step 1: Call the mCase MCP schema tool and show me the raw data shape returned.  
Do not write any UI code yet. I want to confirm the shape before we proceed.
```

This catches data mismatch early — before you have 200 lines of UI to unwind.

3. State Your Assumptions First

Ask Cursor to surface what it's assuming before it acts.

```
Before writing code, list every assumption you're making about:  
- The existing file structure  
- The tech stack and available libraries  
- What the user interaction looks like  
  
Wait for my confirmation before proceeding.
```

Especially useful at the start of a session when context is thin.

4. Non-Goals

Tell Cursor what's out of scope.

Add relationship display to the selected datalist panel.

Non-goals:

- Do not change the datalist list component
- Do not add navigation to related datalists (not in scope yet)
- Do not add new CSS files; use existing styles

Prevents scope creep from a helpful but overzealous agent.

5. Checkpoint Sequence

Give work in steps and stop for review.

Implement field detail display in this order:

1. Type badges based on addType – pause for my review
2. Picklist values for picklist fields – pause for my review
3. Required field indicators – pause for my review

Each pause is a review gate. You catch problems before they compound.

6. Reference Existing Patterns

Point Cursor to what already works.

Add a relationship panel to the datalist detail view.

Follow the same pattern used in the field list component – same layout, same data-fetching approach, same loading state handling.

Reduces drift. New code fits the existing codebase.

7. Confirm Before Deleting or Replacing

When refactoring, make Cursor show you the plan.

Before making any changes to the existing field rendering code, show me what you plan to change and why. Wait for my approval before modifying any existing files.

Prevents well-intentioned rewrites that break working code.

Patterns to Avoid

Anti-pattern	Why it fails
"Make it better"	No definition of better — Cursor guesses
"Refactor this" with no scope	Cursor may rewrite things you didn't ask about
"Fix the bug" with no description	Cursor may "fix" symptoms, not causes
Long conversations for multiple features	Context fills, quality degrades — start fresh per feature
Accepting the first output	Review is part of the workflow, not optional